SKILLNAVORA · AI TOOLS · REPLIT TUTORIAL

# Replit: Complete Tutorial

Build and deploy apps from your browser with Replit and Replit Agent: workspace, prompting, databases and secrets, deployments, GitHub, templates and a project.

Edition 2026 · skillnavora.in

# How to use this guide

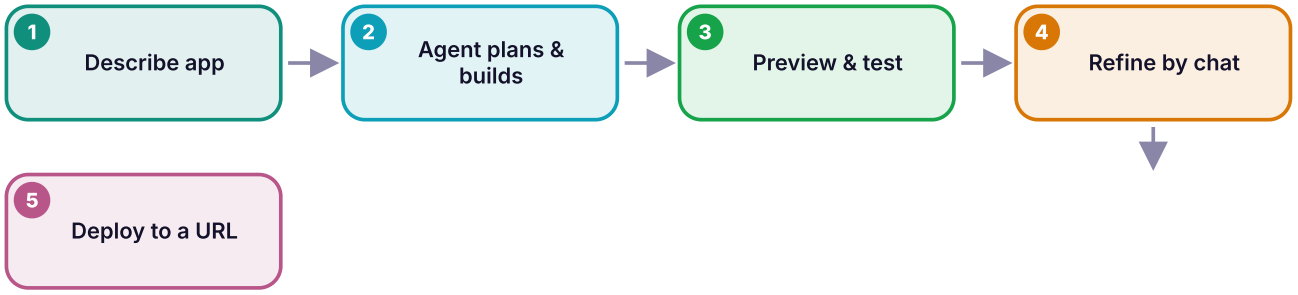
Each class has goals, concepts with Indian examples, key takeaways and a practice exercise, from basic to advanced. Finish with the final project to build something you can show in interviews.

| | Class | Level |
|---|---|---|
| Class 1 | **Replit basics and the workspace** | Basic |
| Class 2 | **Build your first app with Agent** | Basic |
| Class 3 | **Data, secrets and integrations** | Intermediate |
| Class 4 | **Deploy, domains and GitHub** | Intermediate |
| Class 5 | **Prompt templates** | Intermediate |
| Class 6 | **Prompting the Agent like a product manager** | Intermediate |
| Class 7 | **Three practical builds: internal tool, landing page with leads, and an API or bot** | Intermediate |
| Class 8 | **Data, authentication and integrations in detail** | Intermediate |
| Class 9 | **Quality, security and performance checklist** | Advanced |
| Class 10 | **Git, collaboration, environments and handing over to engineers** | Advanced |
| Class 11 | **When to use Replit vs Lovable vs other tools, and PM use cases** | Advanced |
| Class 12 | **Troubleshooting and FAQ: Replit** | Advanced |
| Class 13 | **Practice challenges with solutions: Replit** | Advanced |
| Class 14 | **Project, safety and cost** | Advanced |
| | **Toolkit:** templates, replit cheat sheet, glossary, 12 interview questions, further reading | |

BASIC · CLASS 1

# Replit basics and the workspace

Get set up and understand the layout.




Replit Agent flow

- Sign up at replit.com → **Create App** → describe what you want, or start from a template.
- **Workspace:** file tree, code editor, **Preview** (your running app), **Console/Shell**, **Secrets** (private keys), **Git** panel, **Agent** chat.
- Plans, credits and agent usage change often: check the pricing page before relying on limits.

## Workspace map

| Area | What it is for |
|---|---|
| Agent / chat panel | Describe what you want; the Agent plans, writes code, installs packages and fixes errors |
| File tree and editor | See and edit the code; read what the Agent changed |
| Preview / webview | The running app, in a browser frame |
| Shell / console | Run commands and see logs |
| Secrets | Store keys and passwords securely as environment variables |
| Database and storage tools | Built-in database and file storage options (check current names) |
| Version control / Git | Commits, branches, GitHub link |
| Deployments | Publish the app to a public URL |

**Check before you build:** app names, plan limits, prices and menu labels change often. Use this lesson for the method and confirm the current screens and limits in the official docs.

**Key takeaways**

- Everything runs in the browser.
- Agent chat builds; Preview shows.
- Keep keys in Secrets.

**Practice exercise:** Create an app from a template and find Preview, Secrets and Shell.

BASIC · CLASS 2

# Build your first app with Agent

From prompt to running app.

1. Prompt with a mini-PRD: *"Build a web app 'Class Booking' for a yoga studio: home, class list, book a slot (name, phone), my bookings. Calm green style. Save bookings in a database."*
2. Review the Agent's plan; approve; watch files being created.
3. Use **Preview**; click through; note bugs.
4. Fix by chat: *"The booking form doesn't clear after saving. Fix and add a success message."* One change at a time.
5. Use the **checkpoints/history** to roll back if something breaks.

## A mini-PRD prompt that works

```
Build a leave-request web app for a 30-person company.
Users: Employee (submit, see status), Manager (approve or reject), HR (see all, export CSV).
Pages: Login, My requests, New request, Team requests (manager), Admin (HR).
Data: users(name, email, role), requests(user, type, start, end, reason, status).
Rules: end date after start; managers see only their team; email on decision.
Style: clean, mobile-friendly, blue accent.
Start with the data model and the employee flow only. Ask me before adding anything else.
```

Ask the Agent to **plan first**, review the plan, then let it build in small steps, checking the preview after each step.

**Key takeaways**

- Start with a mini-PRD.
- One change per prompt.
- Roll back with history.

**Practice exercise:** Build the booking app and fix 3 issues by chat.

INTERMEDIATE · CLASS 3

# Data, secrets and integrations

Add a database, API keys and services.

- **Database:** ask for a database (Replit offers built-in Postgres/key-value options; check current names); ask the Agent to create tables and CRUD screens.
- **Secrets:** add API keys in the Secrets tab; the code reads them as environment variables; never paste keys in code or chat.
- **Integrations:** email (Resend/SendGrid), payments (Razorpay/Stripe test mode), AI APIs (Claude, OpenAI, Gemini), auth (login with email/Google).

```
# reading a secret in Python
import os; key = os.environ["RESEND_API_KEY"]
```

## Where each kind of data goes

| Data | Where |
|---|---|
| API keys, passwords | Secrets (environment variables) |
| Users, records, orders | Database (SQL such as PostgreSQL for structured data) |
| Uploaded files and images | Object storage |
| Small settings | Config file or database table |
| Temporary cache | In memory or a cache service |

**Key takeaways**

- Database for saved data.
- Secrets for keys.
- Use test mode for payments.

**Practice exercise:** Add a database and one API key (in Secrets) to your app.

INTERMEDIATE · CLASS 4

# Deploy, domains and GitHub

Publish and keep the code safe.

1. Click **Deploy** and choose the type (autoscale/static/reserved VM: pick the simplest that fits); set run command and secrets; deploy.
2. **Custom domain:** in the deployment settings, add your domain and copy the DNS records to your domain provider.
3. **GitHub:** use the Git panel to connect/push the repo so the code lives outside Replit and teammates can review.
4. **Monitor:** check logs and usage after launch.

## Deployment types (confirm current options)

| Type | Good for |
|---|---|
| Static | Landing pages, portfolios; no server code |
| Autoscale | Web apps and APIs with variable traffic; pay for usage |
| Reserved VM | Always-on services, bots, long processes |
| Scheduled | Jobs that run on a timetable (daily report, cleanup) |

**Key takeaways**

- Deploy gives a public URL.
- Custom domain via DNS records.
- Push to GitHub for safety.

**Practice exercise:** Deploy your app and push it to GitHub.

INTERMEDIATE · CLASS 5

# Prompt templates

Reusable prompts for common builds.

| Build | Prompt starter |
|---|---|
| Landing page + form | "Create a one-page site for [business] with hero, 3 services, pricing, testimonials and a contact form that saves to a database." |
| Dashboard | "Build a dashboard that reads sales.csv and shows revenue by month, top products and a filter by region." |
| AI app | "Build a chat page that answers questions from the text in docs/ using the Claude API (key in Secrets); show sources." |
| Internal tool | "Create a login-protected admin page to approve/reject requests stored in the database." |

**Key takeaways**

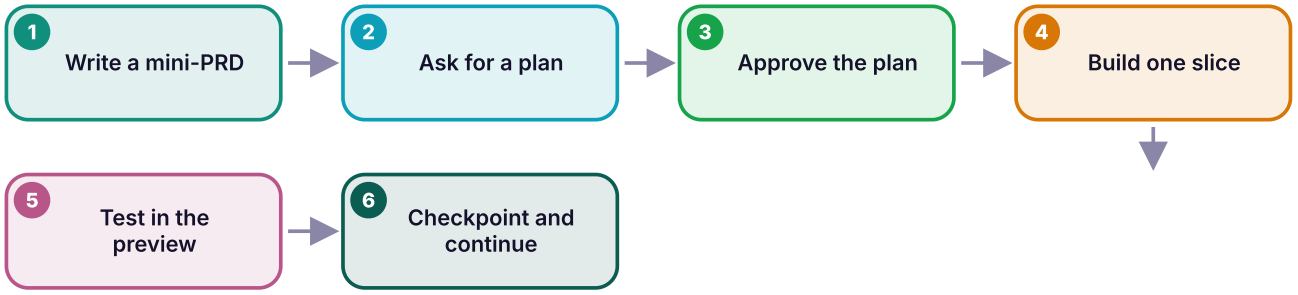
- Name pages, data and style.
- Ask for sample data.
- Ask for tests on key flows.

**Practice exercise:** Use two templates to build mini apps.

INTERMEDIATE · CLASS 6

# Prompting the Agent like a product manager

Write specs, checkpoints and reviews that get reliable results.




A reliable Agent workflow

- **Describe outcomes and rules, not code:** users, pages, data, rules, edge cases, visuals.
- **Slice the work:** authentication, then data model, then main flow, then polish. Large vague prompts create tangled apps.
- **Give examples:** sample records, example emails, screenshots.
- **Say what must not change:** 'Do not change the login page'.
- **Debugging prompt:** 'When I click Submit on the New Request page, nothing happens. The console shows [error]. Find the cause, explain it, then fix it.'
- **Use checkpoints:** restore if the Agent breaks something; commit working versions to Git.
- **Ask for explanations:** 'Summarise what you changed and any risks.'

| Weak prompt | Strong prompt |
|---|---|
| Make a nice dashboard | Create a dashboard with 3 cards (open requests, approved this month, average response time) and a table of requests with filters by status and date |
| Fix the bug | Describe steps, expected vs actual result, error text, and what changed just before |
| Add payments | Add a Razorpay hosted checkout for the Pro plan at Rs 799, store the payment ID, and mark the user as Pro only after the webhook confirms |

**Key takeaways**

- Slice the work and review each slice.
- Write rules and edge cases in the prompt.
- Use checkpoints and Git for safety.

**Practice exercise:** Write a mini-PRD prompt and a debugging prompt for an app idea of your own.

INTERMEDIATE · CLASS 7

# Three practical builds: internal tool, landing page with leads, and an API or bot

Build examples PMs actually need.

## Build 1: Internal request tracker

1. Prompt the data model (requests, users, roles).
2. Add login and roles (employee, manager).
3. Build the form, list and approval flow.
4. Add email or Slack notification via a secret key.
5. Test with two accounts; deploy as Autoscale.

## Build 2: Landing page with waitlist

1. Create a one-page site with hero, features, FAQ and sign-up form.
2. Store sign-ups in the database; prevent duplicates by email.
3. Send a confirmation email; add a privacy notice and consent checkbox.
4. Add analytics events (page view, sign-up).
5. Deploy as Static or Autoscale; connect a custom domain.

## Build 3: Small API or bot

1. Define endpoints (for example GET /status, POST /order).
2. Add authentication via an API key in Secrets.
3. Log requests; add rate limiting.
4. Deploy on a reserved VM if it must stay always on (for example a Telegram or WhatsApp-style bot using the provider's API).

| Prototype goal | What 'good enough' means |
|---|---|
| Validate an idea with 10 users | Works for happy path, basic login, real data, clear failure messages |
| Internal tool for a team | Secure login, backups, error alerts, owner assigned |
| Demo to stakeholders | Polished main flow, sample data |

**Key takeaways**

- Pick builds that match the goal.
- Add auth, validation and notifications deliberately.
- Define 'good enough' before building.

**Practice exercise:** Build one of the three apps and write a list of what is missing for production.

INTERMEDIATE · CLASS 8

# Data, authentication and integrations in detail

Design the data and logins properly.

| Topic | Guidance |
|---|---|
| Data model | List entities (users, orders), fields and relationships; ask the Agent to create tables and migrations; avoid changing structures casually once real data exists |
| Authentication | Use a built-in or well-known auth service; support password reset; consider login with Google; never build your own crypto |
| Authorisation | Check roles and ownership on the server for every request |
| Environment separation | Use separate databases or keys for testing and production |
| Integrations | Use official SDKs and API keys in Secrets; handle errors and rate limits; verify webhooks (signatures) |
| Email and notifications | Use a transactional email provider; test with a sandbox |
| Payments | Use hosted checkout and webhooks; do not trust the browser alone to say a payment succeeded |
| Files | Limit types and sizes; scan or restrict public access |

**Migration tip:** before changing a database structure with real data, ask the Agent to write a migration, test it on a copy, and keep a backup.

**Key takeaways**

- Model data first; protect it with auth and roles.
- Separate test and production.
- Confirm payments server-side.

**Practice exercise:** Draw the data model and roles for an app idea, then prompt the Agent to implement only that.

ADVANCED · CLASS 9

# Quality, security and performance checklist

Make your app safe enough to share.

| Check | Why | How |
|---|---|---|
| Secrets never in code or chat | Keys in prompts or repos leak and get abused | Use the Secrets panel or environment variables; rotate any key you pasted by mistake |
| Authentication and roles | Anyone with the link could see private data | Require login for private pages; check roles on the server, not only in the UI |
| Input validation | Bad input causes errors and attacks | Validate types and lengths; escape output; use parameterised queries |
| Row-level or per-user data rules | Users must only see their own data | Define rules in the database; test with two accounts |
| Rate limits and abuse control | Bots can run up costs | Limit sign-ups, form posts and AI calls; add CAPTCHA if needed |
| Payment handling | Card data is regulated | Use a hosted checkout (for example Razorpay, Stripe); never store card numbers |
| Privacy and consent | Personal data laws apply (DPDP Act) | Collect minimum data, show a privacy notice, allow deletion |
| Backups and recovery | Mistakes and bad deploys happen | Use checkpoints, version control and database backups; test a restore |

## Quality and performance

- **Testing:** ask the Agent to write tests for key flows; run them before deploying; test on a phone.
- **Error handling:** friendly messages, logs, alerts for failures.
- **Performance:** avoid loading everything at once; paginate lists; compress images; check load time on 4G.
- **Accessibility:** labels, contrast, keyboard use.
- **Observability:** view logs and add basic analytics and uptime checks.

**Key takeaways**

- Security basics are mandatory before sharing.
- Test with two users and on a phone.
- Log errors and watch costs.

**Practice exercise:** Run the checklist on one of your apps and fix the top five findings.

ADVANCED · CLASS 10

# Git, collaboration, environments and handing over to engineers

Work with developers and move beyond a prototype.

- **Git and GitHub:** commit often; use branches for risky changes; connect to a GitHub repository so engineers can review and extend.
- **Collaboration:** invite teammates; agree who prompts the Agent to avoid conflicting changes.
- **Environments:** development, staging, production with separate keys and data.
- **Handoff package:** repository link, README (purpose, how to run), environment variable list (names only), data model diagram, known issues, security review notes and next steps.
- **When to hand off:** real customers, payments, personal data, scale, compliance needs or complex integrations.

| Stage | Owner | Output |
|---|---|---|
| Idea and prototype | PM using Agent | Working prototype and learnings |
| Hardening | Engineer | Tests, security, performance, monitoring |
| Production | Engineering team | Scalable, supported system |

**Key takeaways**

- Use Git from day one.
- Prepare a handoff package.
- Know when to graduate to engineering.

**Practice exercise:** Write the README and handoff list for a Replit app you built.

ADVANCED · CLASS 11

# When to use Replit vs Lovable vs other tools, and PM use cases

Choose the right builder.

| Need | Replit | Lovable | No-code (Zapier / Make) | Engineers |
|---|---|---|---|---|
| Quick web app prototype | Good | Excellent for UI-first apps | Not for UI | Slower |
| Backend logic, APIs, bots | Strong | Possible with Supabase / functions | Workflows only | Strong |
| Polished front-end | Good | Excellent | No | Strong |
| Automation between apps | Possible | Limited | Excellent | Possible |
| Production-grade scale and compliance | Hand off | Hand off | Limited | Best |

## PM use cases

- Prototype to test a concept with real users before asking for engineering time.
- Internal tools: approvals, trackers, dashboards.
- Data explorations and small scripts.
- Demos for stakeholders and investors.
- Learning technical constraints by building.

**Key takeaways**

- Match the tool to the job.
- Prototypes validate; engineers productionise.
- Use building to learn constraints.

**Practice exercise:** Pick a feature idea and decide which tool you would use and why.

ADVANCED · CLASS 12

# Troubleshooting and FAQ: Replit

Fix common problems quickly.

| Problem | Likely cause | Fix |
|---|---|---|
| App works in preview but not after deploy | Missing environment variables or wrong port or deployment type | Add secrets to the deployment; check start command; choose the right type |
| Agent keeps looping on an error | Vague prompt or conflicting changes | Stop; describe the exact error; ask for cause and plan; restore a checkpoint |
| Database connection fails | Wrong URL or secret, or schema mismatch | Check secrets; run migrations; test connection |
| Login not working | Callback URL or session settings | Verify configuration; test in a clean browser |
| Costs rising | Many Agent runs or always-on deployments | Prompt in slices; use Autoscale or scheduled where possible; review usage |
| App slow | Large queries or images | Add pagination, indexes, compress images |

**Debug routine:** reproduce, read the console and logs, give the Agent the exact error and steps, review the fix, test, checkpoint.

**Key takeaways**

- Reproduce first; give exact errors.
- Separate environment variables for deploys.
- Checkpoint working states.

**Practice exercise:** Deploy a small app and deliberately remove a secret to see the failure, then fix it.

ADVANCED · CLASS 13

# Practice challenges with solutions: Replit

Test yourself on realistic tasks.

## Challenge 1

Build a feedback form that saves to a database and lets an admin export CSV.

**Solution:** Prompt the data model and admin login; build form and list; add CSV export; test with two roles; deploy; add a privacy notice.

## Challenge 2

Add a daily email report.

**Solution:** Create a script that queries yesterday's data and sends email via a provider key in Secrets; use a Scheduled deployment at 8 am.

## Challenge 3

Secure an API endpoint.

**Solution:** Require an API key from Secrets; validate input; add rate limiting; log requests; return clear errors.

## Challenge 4

Prepare to hand over to engineers.

**Solution:** Connect GitHub; write README; list env variable names; document the data model and known issues.

**Key takeaways**

- Practise data, auth, scheduling and handoff.
- Keep secrets safe.
- Document for engineers.

**Practice exercise:** Create your own three challenges with solutions.

ADVANCED · CLASS 14

# Project, safety and cost

Ship and maintain a real app.

## Project: booking or lead app

PRD → Agent build → database → email confirmation → deploy → custom domain → GitHub → 10-case test list → 1-page README.

- **Review AI code:** check auth, input validation, and that no secrets are in code.
- **Cost:** watch agent usage and deployment costs; stop unused deployments.
- **Data:** follow DPDP for user data; add a privacy page.

## Cost watch-list

- Agent usage can be billed by work done; keep prompts focused and stop loops.
- Deployments and databases may have separate charges.
- Set budgets or alerts if available and review usage weekly.

**Key takeaways**

- Review what the agent wrote.
- Control usage and deployments.
- Privacy page for user data.

**Practice exercise:** Complete the project and record a 3-minute demo.

TOOLKIT

# Templates

## Replit project brief

| Field | Fill in |
|---|---|
| Mini-PRD prompt | |
| Data and tables | |
| Secrets needed | |
| Integrations | |
| Deployment type + domain | |
| 10 test cases | |

TOOLKIT

# Replit cheat sheet

| Term / tool | Meaning / use |
|---|---|
| **Agent** | AI that plans and builds the app |
| **Preview** | Live view of your running app |
| **Secrets** | Private environment variables |
| **Deployment** | Publishing the app to a public URL |
| **Checkpoint** | Saved version you can roll back to |
| **Template** | Starter project |
| **Prompt shape** | Users, pages, data, rules, edge cases, slice |
| **Safe by default** | Secrets, auth, validation, backups |
| **Handoff** | Repo, README, env names, data model, risks |

# Glossary

| | |
|---|---|
| **Agent** | AI that plans and writes code for you |
| **Checkpoint** | A saved state you can restore |
| **Secrets** | Secure storage for keys and passwords |
| **Autoscale** | Deployment that adjusts to traffic |
| **Reserved VM** | Dedicated always-on server |
| **Migration** | A controlled change to the database structure |
| **Webhook** | A call from another service when an event occurs |
| **Staging** | A test copy of production |

INTERVIEW PREP

# 12 interview questions with answers

**Q1. What is Replit Agent?**
An AI that builds, runs and fixes apps in Replit from plain-English descriptions.

**Q2. Where do API keys go?**
In Secrets (environment variables), never in code or chat.

**Q3. How do you deploy?**
Use Deploy, pick a deployment type, add secrets, and optionally a custom domain.

**Q4. Replit vs Lovable?**
Replit gives a full coding environment with many languages and backends; Lovable focuses on front-end web apps with Supabase and GitHub sync.

**Q5. What is Replit Agent best used for?**
Turning a clear description into a working app quickly: prototypes, internal tools and small services, with a human reviewing and testing.

**Q6. How do you write a good prompt for an AI app builder?**
Describe users, pages, data, rules and edge cases, work in small slices, give examples, say what must not change, and ask for a plan before building.

**Q7. How do you keep secrets safe?**
Store them in Secrets or environment variables, never in code, prompts or repositories, and rotate any key that was exposed.

**Q8. What are the deployment options and when to use them?**
Static for simple sites, Autoscale for apps with variable traffic, Reserved VM for always-on services, and Scheduled for timed jobs (confirm current options).

**Q9. What security checks matter before sharing an AI-built app?**
Secrets, authentication and roles, validation, per-user data rules, rate limits, safe payments, privacy and backups.

**Q10. When should you hand an AI-built app to engineers?**
When it has real customers, payments, personal data, scale or compliance needs, or when logic and integrations become complex.

**Q11. How do you control costs when building with an Agent?**
Keep prompts focused, work in slices, stop loops, watch usage and budgets, and avoid unnecessary rebuilds.

**Q12. How do you debug with an Agent?**
Give steps, expected and actual results and error messages, ask for the cause and an explanation before the fix, and test again.

LEARN MORE

# Further reading and tools

- Replit Docs: https://docs.replit.com/
- Replit Agent docs: https://docs.replit.com/replitai/agent
- Replit Deployments: https://docs.replit.com/category/replit-deployments
- Replit Templates: https://replit.com/templates
- OWASP Top 10 (web security): https://owasp.org/www-project-top-ten/
- Razorpay docs: https://razorpay.com/docs/

© SkillNavora · skillnavora.in · Part of the AI Product Management and GenAI Launchpad programs. Practise with the free AI Tutor, AI PM Toolkit and Practice Lab on skillnavora.in.